

PPL LAB 230905290 WEEK6

Rohan Shenoy 230905290 Roll no 32 CSE D1

Q1] write a program in cuda which performs convolution operation on one dimensional input array N of size width using a mask of size mask_width to produce the resultant one dimensional array P of size width

CODE]

```
#include <stdio.h>

#include <cuda_runtime.h>

__global__ void convolution_1D_basic_kernel(float *N, float *M, float *P,
                                           int Mask_Width, int Width) {

    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < Width) {

        float Pvalue = 0;

        int N_start_point = i - (Mask_Width / 2);

        for (int j = 0; j < Mask_Width; j++) {

            if (N_start_point + j >= 0 && N_start_point + j < Width) {

                Pvalue += N[N_start_point + j] * M[j];

            }

        }

    }

}
```

```

        P[i] = Pvalue;

    }

}

int main() {

    const int Width = 10;

    const int Mask_Width = 3;

    size_t size_N = Width * sizeof(float);

    size_t size_M = Mask_Width * sizeof(float);

    float h_N[Width] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    float h_M[Mask_Width] = {0.5, 1.0, 0.5};

    float h_P[Width];

    float *d_N, *d_M, *d_P;

    cudaMalloc((void**)&d_N, size_N);

    cudaMalloc((void**)&d_M, size_M);

    cudaMalloc((void**)&d_P, size_N);

    cudaMemcpy(d_N, h_N, size_N, cudaMemcpyHostToDevice);

    cudaMemcpy(d_M, h_M, size_M, cudaMemcpyHostToDevice);

```

```

int threadsPerBlock = 256;

int blocksPerGrid = (Width + threadsPerBlock - 1) / threadsPerBlock;

convolution_1D_basic_kernel<<<blocksPerGrid, threadsPerBlock>>>(d_N,
d_M, d_P, Mask_Width, Width);

cudaMemcpy(h_P, d_P, size_N, cudaMemcpyDeviceToHost);

printf("Result array P: ");

for (int i = 0; i < Width; i++) printf("%.1f ", h_P[i]);

printf("\n");

cudaFree(d_N); cudaFree(d_M); cudaFree(d_P);

return 0;
}

```

OUTPUT]

```

● 6CSED1@debian:~/Desktop/pp1_lab/lab6$ nvcc q1.cu
● 6CSED1@debian:~/Desktop/pp1_lab/lab6$ ./a.out
Result array P: 2.0 4.0 6.0 8.0 10.0 12.0 14.0 16.0 18.0 14.5
○ 6CSED1@debian:~/Desktop/pp1_lab/lab6$ █

```

Q2] Write a program in cuda to perform selection sort in parallel

CODE]

```
#include <stdio.h>
#include <cuda.h>
#include <limits.h>

__global__ void findMinKernel(int *arr, int n, int start, int *minVal, int *minIdx)
{
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int index = start + tid;

    if (index < n)
    {
        int value = arr[index];

        int old = atomicMin(minVal, value);

        if (value < old)
        {
            atomicExch(minIdx, index);
        }
    }
}

void selectionSortCUDA(int *h_arr, int n)
{
    int *d_arr;
    cudaMalloc((void**)&d_arr, n * sizeof(int));
    cudaMemcpy(d_arr, h_arr, n * sizeof(int), cudaMemcpyHostToDevice);

    int *d_minVal, *d_minIdx;
    cudaMalloc((void**)&d_minVal, sizeof(int));
    cudaMalloc((void**)&d_minIdx, sizeof(int));

    for (int i = 0; i < n - 1; i++)
    {
        int minVal = INT_MAX;
        int minIdx = i;
    }
}
```

```

        cudaMemcpy(d_minVal, &minVal, sizeof(int), cudaMemcpyHostToDevice);
        cudaMemcpy(d_minIdx, &minIdx, sizeof(int), cudaMemcpyHostToDevice);

        int threads = 256;
        int blocks = (n - i + threads - 1) / threads;

        findMinKernel<<<blocks, threads>>>(d_arr, n, i, d_minVal,
d_minIdx);
        cudaDeviceSynchronize();

        cudaMemcpy(&minIdx, d_minIdx, sizeof(int), cudaMemcpyDeviceToHost);

        int temp1, temp2;
        cudaMemcpy(&temp1, &d_arr[i], sizeof(int), cudaMemcpyDeviceToHost);
        cudaMemcpy(&temp2, &d_arr[minIdx], sizeof(int),
cudaMemcpyDeviceToHost);

        cudaMemcpy(&d_arr[i], &temp2, sizeof(int), cudaMemcpyHostToDevice);
        cudaMemcpy(&d_arr[minIdx], &temp1, sizeof(int),
cudaMemcpyHostToDevice);
    }

    cudaMemcpy(h_arr, d_arr, n * sizeof(int), cudaMemcpyDeviceToHost);

    cudaFree(d_arr);
    cudaFree(d_minVal);
    cudaFree(d_minIdx);
}

int main()
{
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);

    selectionSortCUDA(arr, n);

    printf("Sorted array:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

```

```
printf("\n");

return 0;
}
```

OUTPUT]

```
6CSED1@debian:~/Desktop/ppl_lab/lab6$ nvcc q2.cu
6CSED1@debian:~/Desktop/ppl_lab/lab6$ ./a.out
Sorted array:
11 12 22 25 64
```

Q3] Write a code in cuda to perform odd even transportation sort in parallel

CODE]

```
#include <stdio.h>
#include <cuda.h>

__global__ void oddEvenSortKernel(int *arr, int n, int phase)
{
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    int i;

    if (phase % 2 == 0)
        i = 2 * tid;
    else
        i = 2 * tid + 1;

    if (i + 1 < n)
    {
        if (arr[i] > arr[i + 1])
        {
            int temp = arr[i];
            arr[i] = arr[i + 1];
            arr[i + 1] = temp;
        }
    }
}
```

```

}

void oddEvenSortCUDA(int *h_arr, int n)
{
    int *d_arr;
    cudaMalloc((void**)&d_arr, n * sizeof(int));
    cudaMemcpy(d_arr, h_arr, n * sizeof(int), cudaMemcpyHostToDevice);

    int threads = 256;
    int blocks = (n/2 + threads - 1) / threads;

    for (int phase = 0; phase < n; phase++)
    {
        oddEvenSortKernel<<<blocks, threads>>>(d_arr, n, phase);
        cudaDeviceSynchronize();
    }

    cudaMemcpy(h_arr, d_arr, n * sizeof(int), cudaMemcpyDeviceToHost);
    cudaFree(d_arr);
}

int main()
{
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);

    oddEvenSortCUDA(arr, n);

    printf("Sorted array:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    return 0;
}

```

OUTPUT]

```
6CSED1@debian:~/Desktop/pp1_lab/lab6$ nvcc q3.cu
6CSED1@debian:~/Desktop/pp1_lab/lab6$ ./a.out
Sorted array:
11 12 22 25 64
6CSED1@debian:~/Desktop/pp1_lab/lab6$
```

ADDITIONAL QUESTIONS

Q1] Write a program in cuda which takes N integers as input. It converts these integers into their corresponding octal values and stores the result in another array in parallel.

CODE]

```
#include <stdio.h>
#include <cuda.h>

__global__ void decimalToOctal(int *d_input, int *d_output, int n)
{
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    if (tid < n)
    {
        int num = d_input[tid];
        int octal = 0;
        int place = 1;

        while (num > 0)
        {
            int remainder = num % 8;
            octal += remainder * place;
            num = num / 8;
            place *= 10;
        }

        d_output[tid] = octal;
    }
}
```

```

int main()
{
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int h_input[n], h_output[n];

    printf("Enter %d integers:\n", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &h_input[i]);

    int *d_input, *d_output;

    cudaMalloc((void**)&d_input, n * sizeof(int));
    cudaMalloc((void**)&d_output, n * sizeof(int));

    cudaMemcpy(d_input, h_input, n * sizeof(int), cudaMemcpyHostToDevice);

    int threads = 256;
    int blocks = (n + threads - 1) / threads;

    decimalToOctal<<<blocks, threads>>>(d_input, d_output, n);
    cudaDeviceSynchronize();

    cudaMemcpy(h_output, d_output, n * sizeof(int),
cudaMemcpyDeviceToHost);

    printf("\nOctal values:\n");
    for (int i = 0; i < n; i++)
        printf("%d -> %d\n", h_input[i], h_output[i]);

    cudaFree(d_input);
    cudaFree(d_output);

    return 0;
}

```

OUTPUT]

```
6CSED1@debian:~/Desktop/ppl_lab/lab6$ nvcc aq1.cu
6CSED1@debian:~/Desktop/ppl_lab/lab6$ ./a.out
Enter number of elements: 3
Enter 3 integers:
12
123
24

Octal values:
12 -> 14
123 -> 173
24 -> 30
6CSED1@debian:~/Desktop/ppl_lab/lab6$
```

Q2] Write a program in CUDA which takes N binary numbers as input and stores the ones complement of each element in another array in parallel.

CODE]

```
#include <stdio.h>
#include <cuda.h>

__device__ int onesComplement(int num)
{
    int result = 0;
    int place = 1;

    while (num > 0)
    {
        int bit = num % 10;

        if (bit == 0)
            result += 1 * place;
        else
            result += 0 * place;

        num = num / 10;
        place *= 10;
    }
}
```

```

    return result;
}

__global__ void onesComplementKernel(int *d_input, int *d_output, int n)
{
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    if (tid < n)
    {
        d_output[tid] = onesComplement(d_input[tid]);
    }
}

int main()
{
    int n;

    printf("Enter number of binary elements: ");
    scanf("%d", &n);

    int h_input[n], h_output[n];

    printf("Enter %d binary numbers:\n", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &h_input[i]);

    int *d_input, *d_output;

    cudaMalloc((void**)&d_input, n * sizeof(int));
    cudaMalloc((void**)&d_output, n * sizeof(int));

    cudaMemcpy(d_input, h_input, n * sizeof(int), cudaMemcpyHostToDevice);

    int threads = 256;
    int blocks = (n + threads - 1) / threads;

    onesComplementKernel<<<blocks, threads>>>(d_input, d_output, n);
    cudaDeviceSynchronize();
}

```

```

    cudaMemcpy(h_output, d_output, n * sizeof(int),
cudaMemcpyDeviceToHost);

    printf("\nOne's Complement:\n");
    for (int i = 0; i < n; i++)
        printf("%d -> %d\n", h_input[i], h_output[i]);

    cudaFree(d_input);
    cudaFree(d_output);

    return 0;
}

```

OUTPUT]

```

6CSED1@debian:~/Desktop/pp1_lab/lab6$ nvcc aq2.cu
6CSED1@debian:~/Desktop/pp1_lab/lab6$ ./a.out
Enter number of binary elements: 3
Enter 3 binary numbers:
1010
0101
1100

One's Complement:
1010 -> 101
101 -> 10
1100 -> 11
6CSED1@debian:~/Desktop/pp1_lab/lab6$

```